

```
[2]: # Import Libraries
import numpy as np
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

Load the Dataset

```
[3]: data = load_breast_cancer()

X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)

y = pd.Series(data.target, name="target")

print("Dataset shape:", X.shape)
print("\nFirst 5 rows:")
display(X.head())
```

```
[4]: X.shape
```

```
[4]: (569, 30)
```

Check the Dataset

```
[5]: print("Number of samples:", X.shape[0])
print("Number of features:", X.shape[1])

print("\nTarget distribution:")
print(y.value_counts())

print("\nMissing values:")
print(X.isnull().sum().sum())

Number of samples: 569
Number of features: 30

Target distribution:
target
1    357
0    212
Name: count, dtype: int64

Missing values:
0
```

Train-Test Split

```
[6]: X_train, X_test, y_train, y_test = train_test_split(
      X,
      y,
      test_size=0.2,
      random_state=42,
      stratify=y
    )

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

Training samples: 455

Testing samples: 114

Feature Scaling

```
[7]: scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

print("Training data shape:", X_train_scaled.shape)
print("Testing data shape:", X_test_scaled.shape)
```

Training data shape: (455, 30)

Testing data shape: (114, 30)

Baseline Logistic Regression

Before using PSO, we should train Logistic Regression using all 30 features.

```
[8]: baseline_model = LogisticRegression(max_iter=2000)

baseline_model.fit(
    X_train_scaled,
    y_train
)

baseline_pred = baseline_model.predict(
    X_test_scaled
)

baseline_accuracy = accuracy_score(
    y_test,
    baseline_pred
)

print("Baseline Accuracy:",
      baseline_accuracy)
```

Baseline Accuracy: 0.9824561403508771

Define the PSO Fitness Function

```
[10]: def fitness_function(position):

    # Convert continuous position to binary selection
    selected = position > 0.5

    # At least one feature must be selected
    if not np.any(selected):
        return 0

    X_selected = X_train_scaled[:, selected]

    model = LogisticRegression(max_iter=2000)

    model.fit(X_selected, y_train)

    predictions = model.predict(X_selected)

    accuracy = accuracy_score(
        y_train,
        predictions
    )

    return accuracy
```

Better Fitness Function Using Cross-Validation

```
[11]: from sklearn.model_selection import cross_val_score
```

```
[12]: def fitness_function(position):  
  
    selected = position > 0.5  
  
    if not np.any(selected):  
        return 0  
  
    X_selected = X_train_scaled[:, selected]  
  
    model = LogisticRegression(max_iter=2000)  
  
    scores = cross_val_score(  
        model,  
        X_selected,  
        y_train,  
        cv=5,  
        scoring='accuracy'  
    )  
  
    return scores.mean()
```

▼ Understanding the Fitness Function

Suppose a particle is:

[1 0 1 1 0 1]

PSO interprets this as:

F1 → selected F2 → not selected F3 → selected F4 → selected F5 → not selected F6 → selected

The fitness function trains Logistic Regression using:

F1, F3, F4, F6

Suppose cross-validation gives:

Accuracy = 0.956

Then:

Fitness = 0.956

A higher fitness is better.

Initialize PSO

```
[13]: n_particles = 15
      n_features = X_train_scaled.shape[1]

      n_iterations = 30

      w = 0.7
      c1 = 1.5
      c2 = 1.5

      np.random.seed(42)

      positions = np.random.rand(
          n_particles,
          n_features
      )

      velocities = np.random.uniform(
          -1,
          1,
          size=(n_particles, n_features)
      )
```

Here:

`n_particles` `n_particles = 15`

means we have 15 candidate solutions.

`n_iterations` `n_iterations = 30`

means PSO will search for 30 iterations.

`w` `w = 0.7`

is the inertia weight.

It controls how much the previous velocity influences the new velocity.

`c1` `c1 = 1.5`

controls the particle's attraction toward its own best solution.

`c2` `c2 = 1.5`

controls the particle's attraction toward the swarm's best solution.

Initialize Personal and Global Best

```
[15]: pbest_positions = positions.copy()

pbest_scores = np.array([
    fitness_function(position)
    for position in positions
])

best_index = np.argmax(pbest_scores)

gbest_position = pbest_positions[
    best_index
].copy()

gbest_score = pbest_scores[
    best_index
]

print("Initial Best Fitness:",
      gbest_score)

Initial Best Fitness: 0.9780219780219781
```

```
[16]: history = []

for iteration in range(n_iterations):

    for i in range(n_particles):

        # Random values
        r1 = np.random.rand(n_features)
        r2 = np.random.rand(n_features)

        # Update velocity
        velocities[i] = (
            w * velocities[i]
            + c1 * r1 *
              (pbest_positions[i] - positions[i])
            + c2 * r2 *
              (gbest_position - positions[i])
        )

        # Update position
        positions[i] = (
            positions[i] + velocities[i]
        )

        # Keep position between 0 and 1
        positions[i] = np.clip(
            positions[i],
            0,
            1
        )
```

```

# Calculate fitness
score = fitness_function(
    positions[i]
)

# Update personal best
if score > pbest_scores[i]:

    pbest_scores[i] = score

    pbest_positions[i] = (
        positions[i].copy()
    )

# Update global best
if score > gbest_score:

    gbest_score = score

    gbest_position = (
        positions[i].copy()
    )

history.append(gbest_score)

print(
    f"Iteration {iteration + 1:02d} "
    f"| Best Fitness = {gbest_score:.4f}"
)

```

Extract the Best Features

```

[18]: best_features_mask = gbest_position > 0.5

selected_feature_names = X.columns[
    best_features_mask
]

print("Selected Features:")
for feature in selected_feature_names:
    print(feature)

print("\nNumber of selected features:",
      len(selected_feature_names))

print("Best CV Accuracy:",
      gbest_score)

```

Train Final Logistic Regression

```
[19]: X_train_pso = X_train_scaled[
        :, best_features_mask
    ]

    X_test_pso = X_test_scaled[
        :, best_features_mask
    ]

    final_model = LogisticRegression(
        max_iter=2000
    )

    final_model.fit(
        X_train_pso,
        y_train
    )

    final_pred = final_model.predict(
        X_test_pso
    )
```

Evaluate the Final Model

```
[20]: accuracy = accuracy_score(
        y_test,
        final_pred
    )

    precision = precision_score(
        y_test,
        final_pred
    )

    recall = recall_score(
        y_test,
        final_pred
    )

    f1 = f1_score(
        y_test,
        final_pred
    )

    print("PSO + Logistic Regression")
    print("-----")
    print("Accuracy :", accuracy)
    print("Precision:", precision)
    print("Recall   :", recall)
    print("F1 Score :", f1)
```

```
[21]: cm = confusion_matrix(
        y_test,
        final_pred
    )

    print("Confusion Matrix:")
    print(cm)
```

```
Confusion Matrix:
[[41  1]
 [ 4 68]]
```

```
[22]: import matplotlib.pyplot as plt
    from sklearn.metrics import ConfusionMatrixDisplay

    ConfusionMatrixDisplay(
        confusion_matrix=cm
    ).plot()

    plt.title("PSO + Logistic Regression")
    plt.show()
```

Compare Before and After PSO

This is an important part of the practical.

```
[23]: print("Performance Comparison")
    print("-----")

    print("All Features:")
    print("Number of Features:", X.shape[1])
    print("Accuracy:", baseline_accuracy)

    print("\nPSO Selected Features:")
    print("Number of Features:",
          len(selected_feature_names))
    print("Accuracy:", accuracy)
```

```
[24]: plt.figure(figsize=(8, 5))

plt.plot(
    range(1, len(history) + 1),
    history,
    marker='o'
)

plt.xlabel("Iteration")
plt.ylabel("Best Fitness")
plt.title("PSO Convergence")
plt.grid(True)

plt.show()
```